COMP 2402 AB - Fall 2021
Assignment #1

Due: Sunday September 26, 23:59

Submit early and often. No late submissions will be accepted.

Academic Integrity

You may:

- Discuss general approaches with course staff,
- Discuss general approaches with your classmates,
- Use code and/or ideas from the textbook,
- Use a search engine / the internet to look up basic Java syntax,
- Send your code / screen share your code with course staff (although it is unlikely the course staff has the bandwidth to look at individuals' code, unfortunately.)

You may not:

- Send or otherwise share code or code snippets with classmates,
- Use code not written by you, unless it is code from the textbook (and you should cite it in comments),
- Use a search engine / the internet to look up approaches to the assignment,
- Use code from previous iterations of the course, unless it was solely written by you,
- Use the internet to find source code or videos that give solutions to the assignment.

If you ever have any questions about what is or is not allowable regarding academic integrity, please do not hesitate to reach out to course staff. We will be happy to answer. Sometimes it is difficult to determine the exact line, but if you cross it the punishment is severe and out of our hands. Any student caught violating academic integrity, whether intentionally or not, will be reported to the Associate Dean and be penalized as follows:

- First offence: F in the course.
- Second offence: One-year suspension from program.
- Third offence: Expulsion from the University.

These are standard penalties. More-severe penalties will be applied in cases of egregious offences. For more information, please see Carleton University's Academic Integrity page.

Grading

This assignment will be tested and graded by a computer program (and **you can submit as many times as you like, your highest grade is recorded**). For this to work, there are some important rules you must follow:

- Keep the directory structure of the provided zip file. If you find a file in the subdirectory `comp2402a1` leave it there.
- Keep the package structure of the provided zip file. If you find a package `comp2402a1;` directive at the top of a file, leave it there.
- Do not rename or change the visibility of any methods already present. If a method or class is public leave it that way.
- Do not change the `main(String)` method of any class. Some of these are setup to read command line arguments and/or open input and output files. Don't change this behaviour. Instead, learn how to use command-line arguments to do your own testing.
- Submit early and often. The submission server compiles and runs your code and gives you a mark. You can submit as often as you like and only your latest submission will count. There is no excuse for submitting code that does not compile or does not pass tests.
- Write efficient code. The submission server places a limit on how much time it will spend executing your code, even on inputs with a million lines. For some questions it also places a limit on how much memory your code can use. If you choose and use your data structures correctly, your code will easily execute within the time limit. Choose the wrong data structure, or use it the wrong way, and your code will be too slow for the submission server to grade (resulting in a grade of 0).

Submitting and Server Tests

The submission server is ready: here. If you have issues, please post to Discord to the teaching team (or the class) and we'll see if we can help.

When you submit your code, the server runs tests on your code. These are bigger and better tests than the small number of tests provided in the "Local Tests" section further down this document. They are obfuscated from you, because you should try to find exhaustive tests of your own code. This can be frustrating because you are still learning to think critically about your own code, to figure out where its weaknesses are. But have patience with yourself and make sure you read the question carefully to understand its edge cases.

Warning: Do not wait until the last minute to submit your assignment. There is a hard 5 second limit on the time each test has to complete. If the server is heavily loaded, borderline tests may start to fail. **You can submit multiple times and your best score is recorded.**

The Assignment

Part A: The Setup

Start by downloading and decompressing the Assignment 1 Zip File (comp2402a1.zip), which contains a skeleton of the code you need to write. You should have the files: Part0.java, Part1.java, Part2.java, Part3.java, Part4.java, Part5.java, MyArrayStack.java, MyList.java and Tester.java.

The skeleton code in the zip file compiles fine. Here's what it looks like when you unzip, compile, and run Part0 from the command line:

```
alina@euclid:~$ unzip comp2402a1.zip
Archive:  comp2402a1.zip
  inflating: comp2402a1/MyArrayStack.java
  inflating: comp2402a1/MyList.java
  inflating: comp2402a1/Part0.java
  inflating: comp2402a1/Part1.java
  inflating: comp2402a1/Part2.java
  inflating: comp2402a1/Part3.java
  inflating: comp2402a1/Part4.java
  inflating: comp2402a1/Part5.java
  inflating: comp2402a1/Tester.java
alina@euclid:~$ javac comp2402a1/*.java
alina@euclid:~$ java comp2402a1.Part0
blue
red
yellow
violet
[Hit Ctrl-d or Ctrl-z to end the input here]
blue
red
yellow
violet
Execution time: 8.128651
alina@euclid:~$
```

You can also run the Tester program:

```
alina@euclid:~$ java comp2402a1.Tester

Test AddToBack------
[Ljava.lang.Object;@5acf9800
[Ljava.lang.Object;@5acf9800
```

```
[Ljava.lang.Object;@4617c264
[Ljava.lang.Object;@36baf30c
[Ljava.lang.Object;@36baf30c
[Ljava.lang.Object;@7a81197d
[Ljava.lang.Object;@7a81197d
[Ljava.lang.Object;@7a81197d
[Ljava.lang.Object;@7a81197d
[Ljava.lang.Object;@5ca881b5
[Ljava.lang.Object;@5ca881b5
Done Test AddToBack------
Test RemoveFromFront------
[Ljava.lang.Object;@24d46ca6
[Ljava.lang.Object;@24d46ca6
[Ljava.lang.Object;@24d46ca6
[Ljava.lang.Object;@24d46ca6
[Ljava.lang.Object;@24d46ca6
[Ljava.lang.Object;@4517d9a3
[Ljava.lang.Object;@4517d9a3
[Ljava.lang.Object;@372f7a8d
[Ljava.lang.Object;@2f92e0f4
[Ljava.lang.Object;@28a418fc
[Ljava.lang.Object;@5305068a
alina@euclid:~$
```

If you are having trouble running these programs, figure this out first before attempting to do the assignment. If you are stuck, ask on Discord, and course staff or another student will likely help you fairly quickly.

Part B: The Interface

This part is about using the Java Collections Framework to accomplish some basic text-processing tasks. These questions involve choosing the right abstraction (Collection, Set, List, Queue, Deque, SortedSet, Map, or SortedMap) to efficiently accomplish the task at hand. The best way to do these is to read the question and then think about what type of Collection is best to use to solve it. There are only a few lines of code you need to write to solve each of them.

The file `Part0.java` in the zip file actually does something. You can use its `doIt()` method as a starting point for your solutions. Compile it. Run it. Test out the various ways of inputting and outputting data. You should not need to modify `Part0.java`, it is a template.

You can download some sample input and output files for each question as a zip file (`a1-io.zip`). If you find that a particular question is unclear, you can probably clarify it by looking at the sample files for that question.

Unless specified otherwise, "sorted order" refers to the natural sorted order on Strings, as defined by `String.compareTo(s)`.

1. [10 marks] Read the input one line at a time until you have read all lines. Now output these lines in the opposite order from which they were read, but with consecutive duplicates removed.
2. [10 marks] A 2402-block is 2402 consecutive lines. Read in the input one line at a time, as a sequence of some number of 2402 blocks. Now output these blocks in the opposite order from which they were read, but within each block output the lines in the order in which they were read.
3. [10 marks] Read the input one line at a time until you have read all n lines. Now output the minimum line out of the first floor(n/2) lines (where smaller is with respect to the usual order on Strings, as defined by `String.compareTo()`). If n <= 1 you should output the empty string.
4. [10 marks] Read the input one line at a time and output the current line if and only if it is a duplicate of some previous line.
5. [10 marks] Read the input one line at a time and output only the last 2021 lines in the order they appear. If there are fewer than 2021 lines, output them all. For full marks, your code should be fast and should never store more than 2022 lines.

Part C: The Implementation

The (provided) `MyArrayStack` class is an (incomplete) implementation of the `MyList` interface. Your task is to complete the implementation of the `MyArrayStack` class, as specified below. Default implementations are provided for the 3 methods so that you can compile and run `Tester.java` right out of the gate; these defaults will not pass any of the tests, however.

6. [15 marks] Implement
   ```
   public MyList<T> shuffle(MyList<T> other)
   ```
   Return a new `MyArrayStack` that results from alternating individual elements from `this` and `other`. (i.e. `[this[0], other[0], this[1], other[1], …]`)
   If the given lists are not of equal length, just place the extra elements onto the end of the returned `MyArrayStack`.
7. [15 marks] Implement
   ```
   public MyList<MyList<T>> countOff(int n)
   ```
   Return a `MyList` of `n` `MyArrayStacks`, each with every `n`'th entry (as you would "count off" elementary kids into `n` groups). (i.e. the `0`th list contains all elements in `this` with index `0 mod n`, and the `1`st list contains all elements in `this` with index `1 mod n`, and so on.)
   If list `this` is not evenly divisible by `n`, then some lists will be different lengths than others. If list `this` is not big enough to divide into `n` (because size `< n`) then some lists may be empty. If `n = 0` then you should return the empty list.
8. [15 marks] Implement
   ```
   public void reverse()
   ```
   Reverse the `MyArrayStack` without using recursion or too much extra space. If you fail some of the later server tests, it may be because you are using recursion or using too many extra variables (or a small number of large arrays).

You will probably want to write a meaningful `toString` method to start so that you can test and debug the above methods as you go along.

Local Tests

For Parts 1, 2, 3, 4 and 5, you can download some sample input and output files for each question as a zip file (a1-io.zip). Once you get a sense for how to test your code with these input files, write your own tests that test your program more thoroughly (**the provided tests are not exhaustive!**)

For Parts 6, 7, and 8 (`MyArrayStack`), a `Tester.java` is provided with some tests of the provided methods. Since the `toString` method is not meaningful, these tests are gibberish. I recommend you add tests to `Tester.java` to test your `MyArrayStack` locally (as opposed to on the server). We recommend you do this incrementally, that is, bit-by-bit. When you write the `shuffle` method, test it fully before moving on. When you write the `countOff` method, test it fully before moving on. It is much easier to debug when you have confidence that most things are already working.

- Test edge cases, e.g. empty lists, lists of different lengths, etc.
- Use small test cases at first, then go bigger only when necessary.
- Test individual methods as you write them. You can test an incomplete method to make sure it's doing what you're expected thus far.
- You will likely want to write a `toString` method to help debug.

Tips, Tricks, and FAQs

How should I approach each problem?

- Make sure you understand it. Construct small examples, and compute (by hand) the expected output. If you aren't sure what the output should be, go no further until you get clarification.
- Now that you understand what you are supposed to output, and you've been able to solve it by hand, think about how you solved it and whether you could explain it to someone. How about explaining it to a computer?
- If it still seems challenging, what about a simpler case? Can you solve a similar or simplified problem? Maybe a special case? If you were allowed to make certain assumptions, could you do it then? Try constructing your code incrementally, solving the smaller or simpler problems, then, only expanding scope once you're sure your simplified problems are solved.

How should I test my code?

- You should be testing your code as you go along.
- Start small so that you can compute the correct solution by hand.
- Think about edge cases.
- Think about large inputs, random inputs.
- Test for speed.